

Guia de Bordo do Desenvolvedor

Sumário

Guia de Bordo do Desenvolvedor	1
Construindo Soluções com Projetos Reais	3
A Jornada do Construtor: Entendendo o Jogo do PjBL	3
Letramento em IA: Usando seu Copiloto para Acelerar o Aprendizado, Não para Pular a Jornada	5
1. A Regra de Ouro: A IA é sua Ferramenta de Consulta, Não a Solucionadora de Problemas	5
2. O Mecanismo por Trás da Mágica (Versão Rápida)	6
3. Engenharia de Prompts para Desenvolvedores (O Método C.T.P.F. Aplicado)	6
4. O Protocolo de Uso Consciente: Seu Checklist para o Dia a Dia	7
Nossa Jornada em Duas Fases: Fundação e Execução	8
As 7 Atividades Chave do Desenvolvedor	10
Passo 1: Entenda a Missão (Diagnóstico)	10
Passo 2: Crie o Mapa do Tesouro (Requisitos)	11
Passo 3: Trace a Rota (Planejamento)	13
Passo 4: Abasteça suas Ferramentas (Pesquisa)	15
Passo 5: Mão na Massa! (Implementação)	17
Passo 6: Cace os Bugs e Melhore (Validação)	18
Passo 7: Apresente sua Obra e Reflita (Entrega)	19
Do Caminho Reto à Espiral Ágil: Evoluindo seu Processo de Desenvolvimento	21
A Ilusão da Linha Reta	21
Bem-vindo ao Desenvolvimento em Espiral	21
A Espiral Dentro da Espiral: Como os Profissionais Realmente Constroem	22
Como isso Afeta suas Entregas e Avaliações?	22
Guia de Avaliação por Pares: Aprendendo Juntos	24
Nossa Etiqueta de Feedback: Construindo um Ambiente Profissional	24
O que fazer com um feedback inadequado?	25
Por que Avaliar Seus Colegas? A Missão Win-Win	25
Recebendo e Utilizando seu Feedback: O Ciclo de Melhoria Contínua	26
Como Avaliar: Entendendo a Escala e o Poder das Palavras	26
A Escala de Avaliação (Likert):	27
O Feedback Qualitativo (Obrigatório):	27
Passo 1: Entenda a Missão (Diagnóstico)	29
Passo 2: Crie o Mapa do Tesouro (Requisitos)	31
Passo 3: Trace a Rota (Planejamento)	33
Passo 4: Abasteça suas Ferramentas (Pesquisa)	35
Passo 5: Mão na Massa! (Implementação)	37

Passo 6: Cace os Bugs e Melhore (Validação)	39
Passo 7: Apresente sua Obra e Reflita (Entrega)	41

Construindo Soluções com Projetos Reais

Olá, futuro(a) desenvolvedor(a)!

Bem-vindo(a) a uma jornada de aprendizado que vai te transformar em um verdadeiro criador de tecnologia. Para te guiar, usaremos uma metodologia poderosa, conhecida pela sigla **PjBL**. Mas o que ela realmente significa? Prepare-se, pois você vai dominar dois superpoderes de uma só vez.

A Jornada do Construtor: Entendendo o Jogo do PjBL

Você já jogou um videogame de aventura? Se sim, você já entende a essência do PjBL.

Em um jogo, você tem uma **grande missão** (o "main quest"): resgatar alguém, derrotar o chefe final, construir um império. Essa missão é o seu grande objetivo, o que te guia do começo ao fim. Isso é a **Aprendizagem Baseada em Projetos (Project-Based Learning)**. O foco é no grande produto final, na entrega, na construção de algo concreto e funcional.

Mas o que acontece durante a jornada? Você não anda em linha reta até o chefe final. Você encontra portas trancadas, puzzles para resolver, rios que precisam de uma ponte, inimigos com padrões que você precisa decifrar. Cada um desses é um **mini-problema** que exige que você pare, investigue, aprenda uma nova habilidade e aplique-a para poder continuar. Isso é a **Aprendizagem Baseada em Problemas (Problem-Based Learning)**. O foco é na investigação, na descoberta e na superação de desafios específicos.

É aqui que a mágica acontece: **A Aprendizagem Baseada em Projetos (Project-BL) é a sua grande missão, e a Aprendizagem Baseada em Problemas (Problem-BL) é a habilidade que você usa para resolver cada puzzle no caminho.**

Um não existe sem o outro. Seu grande projeto será, na verdade, uma sequência de pequenos problemas que você aprenderá a resolver.

Nossa Abordagem: Foco no Projeto, Movido por Problemas

No nosso programa, o seu foco será sempre a entrega de um **PROJETO** funcional – um site, um aplicativo, uma solução completa. Por isso, chamaremos nossa metodologia principal de **Aprendizagem Baseada em Projetos (PjBL)**.

Mas saiba que, no dia a dia, você estará constantemente no modo "detetive", resolvendo pequenos problemas para fazer seu grande projeto avançar. Quando encontrar um bug que não entende, um erro de CORS, ou a necessidade de implementar uma autenticação segura, você estará praticando o Problem-Based Learning.

É exatamente assim que se trabalha em uma empresa de tecnologia: há um grande projeto a ser entregue (Project-BL), e o dia a dia dos desenvolvedores é uma sucessão de resoluções de problemas técnicos (Problem-BL).

Os 7 passos que mostraremos a seguir são o seu "guia estratégico" para essa jornada, explicando como gerenciar a grande missão e como se equipar para vencer cada desafio no caminho. Mas antes, precisamos falar de uma grande aliada a IA.

Letramento em IA: Usando seu Copiloto para Acelerar o Aprendizado, Não para Pular a Jornada

Antes de mergulharmos no nosso primeiro projeto, precisamos falar sobre a ferramenta mais poderosa e controversa que você terá à sua disposição: a Inteligência Artificial.

Neste curso, você não só pode, como **deve** usar a IA. Pense nela como um **Copiloto** para sua jornada de aprendizado: uma ferramenta para ajudar na navegação e acelerar as partes repetitivas, liberando sua mente para focar no que realmente importa — pilotar a solução e entender a lógica do código.

Este capítulo não é um manual completo de IA, mas um **guia de configuração rápida**. Nosso objetivo é um só: ensinar você a usar a IA para acelerar seu aprendizado, e não para fazer o trabalho por você e te deixar sem saber dirigir.

1. A Regra de Ouro: A IA é sua Ferramenta de Consulta, Não a Solucionadora de Problemas

Imagine que você tem acesso a um assistente de pesquisa ultrarrápido. Ele leu toda a documentação de programação do mundo e pode encontrar qualquer informação em segundos. No entanto, ele não tem contexto sobre o seu projeto específico e não possui a capacidade de tomar decisões de engenharia.

É exatamente assim que você deve tratar a IA.

-  **O Jeito Errado (e Inútil para seu Aprendizado):**
 - "Crie uma aplicação em React que busca um CEP e mostra a previsão do tempo."
 - **Resultado:** A IA vai gerar um bloco de código. Você vai copiar e colar. O projeto vai funcionar, e você não terá aprendido **nada** sobre componentes, estados, hooks ou APIs. Você fraudou seu próprio aprendizado.
-  **O Jeito Certo (O Uso do Profissional):**
 - "Estou construindo uma aplicação em React para buscar um CEP. Qual é a melhor forma de armazenar o valor do CEP digitado pelo usuário no estado do componente?"
 - "Estou usando a API `fetch` para chamar a ViaCEP e estou recebendo um erro de CORS. O que é CORS e quais são as formas comuns de resolvê-lo no desenvolvimento front-end?"
 - "Este é o meu código para a função de busca. Você consegue identificar algum bug ou sugerir uma forma de deixá-lo mais legível e eficiente?"

Percebe a diferença? No jeito certo, **você está no comando**. Você usa a IA como uma ferramenta de consulta para desempacar, aprender conceitos, sugerir boas práticas e revisar seu trabalho. O objetivo do PBL é a **jornada de aprendizado** que acontece durante a resolução do problema. Usar a IA para pular essa jornada é como ir à academia e pedir para outra pessoa levantar os pesos por você: a tarefa é concluída, mas seu músculo (o cérebro) não cresce.

2. O Mecanismo por Trás da Mágica (Versão Rápida)

Para usar a ferramenta sem ser enganado por ela, você precisa entender um único conceito: a IA não "sabe" nada. Ela é, em essência, um **sistema avançado de previsão de texto**.

Pense no corretor automático do seu celular. Se você digita "Feliz", ele sugere "aniversário". Ele não sabe o que é um aniversário; ele apenas calculou que, estatisticamente, essa é a palavra que mais aparece depois de "Feliz". O ChatGPT funciona sob o mesmo princípio, mas em uma escala monumental.

Por que isso é vital? Porque explica a **Alucinação**. Se a IA não sabe a resposta exata para sua pergunta, ela não diz "não sei". Ela inventa uma resposta que *parece* correta, pois a sequência de palavras é estatisticamente plausível. Ela pode inventar uma função que não existe em uma biblioteca ou citar uma documentação falsa com total convicção.

Sua Defesa: Nunca aceite uma informação técnica da IA como verdade absoluta. Se ela sugerir uma nova biblioteca, vá ao Google e verifique se ela existe. Se ela der uma solução de código, teste-a. **Confie, mas verifique.**

3. Engenharia de Prompts para Desenvolvedores (O Método C.T.P.F. Aplicado)

"Instruções vagas geram resultados inúteis". Para extrair respostas de qualidade da IA, você precisa ser específico. Use o método **C.T.P.F.** para estruturar seus pedidos:

- **[C] Contexto:** Dê o cenário. "Estou no Problema 2 do curso, desenvolvendo com React e a API ViaCEP."
- **[T] Tarefa:** O verbo de comando. "Explique o conceito de `useEffect`."
- **[P] Persona:** O papel que a IA deve assumir. "Aja como um Engenheiro de Software Sênior e mentor."
- **[F] Formato:** Como você quer a resposta. "Dê a resposta em tópicos, com exemplos de código comentados."

Exemplo Prático (copie e use!):

[Contexto] Estou aprendendo React e estou com uma dúvida no Problema 2 do meu curso, onde preciso consumir a API da ViaCEP.

[Persona] Aja como um Engenheiro de Software Sênior que é um ótimo mentor para desenvolvedores júnior.

[Tarefa] Explique para que serve o hook `useEffect`. Foque em quando e por que eu deveria usá-lo para fazer uma chamada de API.

[Formato] Apresente a resposta em tópicos simples. Mostre um exemplo de código claro, com comentários explicando cada linha, especialmente o array de dependências.

Compare o resultado deste prompt com o resultado de perguntar apenas "o que é useEffect?". A diferença será brutal.

4. O Protocolo de Uso Consciente: Seu Checklist para o Dia a Dia

1. **Use para Aprender, Não para Copiar:** Sua pergunta para a IA deve ser sobre um conceito, uma boa prática ou um erro, nunca sobre a solução final.
2. **Sempre Verifique:** A IA alucina. Trate cada resposta como um rascunho que precisa ser validado na documentação oficial ou através de testes.
3. **Proteja seus Dados:** Nunca cole dados sensíveis (senhas, chaves de API, informações pessoais) em chats de IA públicos. O que você digita pode ser usado para treinar o modelo.
4. **Assuma a Autoria:** O código final no seu repositório é de sua responsabilidade. Se ele contiver um bug que a IA sugeriu, o bug é seu. Você é o desenvolvedor, a IA é apenas a ferramenta.

Conclusão: Você no Comando

A IA não vai roubar seu emprego, mas um desenvolvedor que a usa com eficiência certamente terá uma vantagem. O profissional do futuro é aquele que une a intuição e o pensamento crítico do humano com a velocidade e a capacidade de pesquisa da máquina.

Use este capítulo como seu guia. A IA é sua aliada mais poderosa nesta jornada, desde que você se lembre sempre de quem é o piloto.

Nossa Jornada em Duas Fases: Fundação e Execução

Para espelhar como as equipes de software de alta performance trabalham, nossa jornada não será uma longa linha reta, mas sim um processo dividido em **duas grandes fases de entrega e avaliação**. Em cada projeto, você irá:

1. **Construir a Fundação:** Planejar com cuidado o "o quê" e o "porquê" do seu projeto.
2. **Executar com Agilidade:** Construir sua solução de forma iterativa e apresentar a jornada completa.

Vamos ver como isso funciona.

Fase 1: Fundação (O Plano de Voo)

Nesta fase, seu foco é transformar a ideia inicial do projeto em um plano claro e bem definido. Você se concentrará nas atividades dos **Passos 1 e 2** (descritos no próximo capítulo).

- **O que você vai fazer?** Analisar o problema, definir a visão, o público-alvo, os objetivos e criar uma lista inicial de requisitos.
- **Qual é o entregável?** Ao final desta fase, você submeterá um único artefato: o seu **Plano de Projeto**. Ele é a combinação do seu Documento de Visão e da sua Lista de Requisitos.
- **Por que uma avaliação aqui?** O feedback que você receberá de seus colegas nesta fase é muito importante. Ele valida se seu plano é sólido, claro e alinhado com o desafio, evitando que você comece a construir na direção errada.

Fase 2: Execução e Entrega (A Expedição)

Com o plano validado, é hora de construir. Esta fase abrange as atividades dos **Passos 3 a 7** (também descritos no próximo capítulo). A grande mudança aqui é que você **não fará uma entrega para cada passo**. Em vez disso, você usará esses passos como um ciclo contínuo de trabalho. Você irá planejar em um quadro, pesquisar, codificar, testar e refinar de forma iterativa, assim como um profissional.

- **O que você vai fazer?** Organizar suas tarefas em um quadro Kanban, pesquisar e documentar decisões, desenvolver o MVP (Minimum Viable Product), testar, fazer o deploy e preparar a comunicação final.
- **Qual é o entregável?** Ao concluir seu MVP, você submeterá um **Pacote de Entrega Final**, que conta a história completa da sua jornada de construção. Este pacote incluirá:
 1. O link para o seu **Quadro de Gerenciamento de Tarefas** finalizado.
 2. O seu **Registro de Decisões Técnicas** completo.
 3. O link para o **Repositório de Código-Fonte** com todo o histórico.

4. O link para a **Aplicação Publicada (Deploy)**.
 5. Sua **Apresentação Final** e a **Documentação README** completa.
- **Por que uma avaliação única no final?** Porque ela permite uma avaliação holística. Seus colegas poderão ver não apenas o produto final, mas *como* você chegou lá: como seu plano evoluiu no quadro Kanban, por que você tomou certas decisões técnicas e como foi seu processo de desenvolvimento através dos commits.

Este modelo espelha como as equipes de alta performance trabalham: planejam com cuidado e executam com agilidade.

As 7 Atividades Chave do Desenvolvedor

A seguir, detalhamos as sete atividades fundamentais que compõem sua jornada.

Passo 1: Entenda a Missão (Diagnóstico)

O que fazer?

Antes de escrever uma linha de código, seja um detetive. Leia o problema com atenção. Quem são os usuários? Qual é a necessidade real deles? Quais são as regras do jogo (restrições de tecnologia, prazos)? Mergulhe no contexto.

Seu objetivo aqui:

Ter uma visão 360° do desafio. Você deve ser capaz de explicar o problema com suas próprias palavras para qualquer pessoa.

💡 Por que isso é importante?

Construir um software incrível que ninguém precisa é um desperdício de talento. Este passo permite que você está resolvendo o problema certo, desenvolvendo uma visão de produto, e não apenas de código.

📦 Artefatos a serem entregues

Um **Documento de Visão do Projeto** (Project Vision Document). Pode ser a seção inicial do seu arquivo README.md. Este documento deve conter um resumo claro do problema, quem são os usuários-alvo, qual o objetivo principal da solução e quais são as restrições (tecnológicas ou de escopo) mais importantes.

- *Este documento é a primeira metade do seu entregável da **Fase 1: Fundação**. Ele será combinado com a Lista de Requisitos do próximo passo para formar o **Plano de Projeto**, que será o seu primeiro artefato submetido para avaliação.*

💡 Dica de Mestre: Faça uma Autoavaliação!

A qualidade do seu Documento de Visão é a base de todo o projeto. **Antes de submeter o seu Plano de Projeto da Fase 1**, use o guia de avaliação correspondente para garantir que esta primeira parte da sua fundação está sólida, clara e alinhada com o desafio.

Passo 2: Crie o Mapa do Tesouro (Requisitos)

O que fazer?

Agora que você entendeu o problema, transforme-o em uma lista de objetivos claros.

- **Requisitos Funcionais:** O que o sistema *deve fazer*? (Ex: "Permitir que o usuário se cadastre").
- **Requisitos Não Funcionais:** Como o sistema *deve ser*? (Ex: "Rápido", "Seguro", "Responsivo").

Seu objetivo aqui:

Criar uma lista que servirá como seu "checklist de sucesso". É o seu guia para saber quando o projeto estará pronto.

💡 Por que isso é importante?

Sem um mapa, qualquer caminho serve. Esta lista evita que você se perca no meio do desenvolvimento e garante que você entregue o que foi proposto.

📦 Artefatos a serem entregues

Uma **Lista de Requisitos (Backlog Inicial)**. Este documento deve separar claramente os Requisitos Funcionais (RFs) – o que o sistema faz – dos Requisitos Não Funcionais (RNFs) – como o sistema deve ser. Formate como uma lista ou tabela para facilitar o acompanhamento. Insira nesta lista como o seu backlog bruto; no próximo passo, você irá refinar cada um desses itens em tarefas de desenvolvimento claras e centradas no usuário.

Ex: RF01: O usuário deve poder se cadastrar. RNF01: A interface deve ser responsiva.

- *Este documento é a segunda e última parte do seu entregável da **Fase 1: Fundação**. Ao combiná-lo com o Documento de Visão, você terá seu **Plano de Projeto** completo, pronto para ser submetido.*

💡 Dica de Mestre: Faça uma Autoavaliação!

O seu Backlog Inicial é o mapa que guiará toda a construção do seu projeto. **Antes de submeter o seu Plano de Projeto da Fase 1**, use o guia de avaliação para verificar se seu mapa está completo, preciso e se responde diretamente à visão que você definiu no passo anterior.

Passo 3: Trace a Rota (Planejamento)

O que fazer?

Sua missão aqui é traduzir a lista de requisitos do Passo 2 em tarefas concretas para o seu quadro Kanban. A técnica mais poderosa para isso é escrever **Histórias de Usuário (User Stories)**. Uma História de Usuário reformula um requisito do ponto de vista de quem vai usar a aplicação, garantindo que você construa com foco no valor para o usuário.

Use o seguinte formato para cada funcionalidade principal:

"Como um(a) <tipo de usuário>, eu quero <realizar uma ação>, para que <eu obtenha um benefício>."

Exemplo: O requisito RF01: O usuário deve poder se cadastrar se transforma na história: **"Como um novo visitante, eu quero criar uma conta usando meu email e senha, para que eu possa salvar minhas transações financeiras."**

Cada História de Usuário se tornará um "cartão" no seu quadro.

Seu objetivo aqui:

Um plano de ação claro, com tarefas priorizadas. Você saberá exatamente por onde começar.



Por que isso é importante?

Isso promove sua autonomia e habilidade de gerenciar o tempo e a complexidade. No mundo real, organização é tão crucial quanto o código.



Artefatos a serem entregues

Um **Quadro de Gerenciamento de Tarefas**. Utilize uma ferramenta como Trello, Jira ou GitHub Projects para criar um quadro Kanban. As colunas devem representar seu fluxo de trabalho (ex: 'A Fazer', 'Em Andamento', 'Concluído'). Os **'cartões' devem ser as Histórias de Usuário** que você escreveu. Se uma história for muito grande, quebre-a em tarefas técnicas menores (ex: "Criar componente de formulário", "Criar endpoint da API").

- *Este quadro é uma ferramenta viva que você usará durante todo o desenvolvimento e, portanto, **não será entregue isoladamente**. O link para a versão finalizada, que conta a história do seu progresso, será incluído no **Pacote de Entrega Final da Fase 2**.*

 Dica de Mestre: Faça uma Autoavaliação!

Seu quadro Kanban é seu painel de controle para a Fase de Execução. Embora ele só seja avaliado no final, um bom planejamento inicial é fundamental. Use o guia de avaliação como um **checklist para garantir que seu quadro esteja bem estruturado desde o início**. Um quadro bem organizado agora tornará seu desenvolvimento na Fase 2 muito mais eficiente.

Passo 4: Abasteça suas Ferramentas (Pesquisa)

O que fazer?

Você sabe *o que* precisa construir e tem um plano. Agora, descubra *como*. Esta é a hora de agir como um verdadeiro pesquisador, usando todas as ferramentas à sua disposição:

- **Fontes Tradicionais:** Mergulhe na **documentação oficial** (sua principal fonte da verdade), assista a **tutoriais** para ver a prática em ação, leia **artigos** para aprofundar conceitos e experimente pequenos trechos de código para validar seu entendimento.
- **Seu Copiloto de IA:** Use a Inteligência Artificial como seu assistente de pesquisa pessoal para acelerar o processo. Faça as perguntas certas, conforme aprendeu no Capítulo 0:
 - **Para entender conceitos:** "Explique o que é JWT e como ele se difere de uma sessão baseada em cookies."
 - **Para comparar tecnologias:** "Quais são os prós e contras de usar Prisma vs. Sequelize em um projeto com Node.js e PostgreSQL?"
 - **Para obter exemplos de sintaxe:** "Mostre-me um exemplo básico de como usar a API `fetch` dentro de um `useEffect` em React para buscar dados."

Lembre-se: a IA acelera sua pesquisa, mas a documentação oficial é sempre a autoridade final. **Confie, mas verifique.**

Seu objetivo aqui:

Adquirir o conhecimento técnico específico para resolver este problema.

💡 Por que isso é importante?

É aqui que a mágica acontece. Você aprende a aprender de forma autônoma. Esta é a habilidade mais valiosa de um desenvolvedor no mercado de tecnologia, que está sempre em mudança.

📦 Artefatos a serem entregues

Um **Registro de Decisões Técnicas**. Pode ser um documento simples (`decisions.md`) no seu repositório. Para cada decisão técnica importante (ex: 'Escolher a biblioteca X para fazer requisições HTTP', 'Usar JWT para autenticação'), anote a decisão, a justificativa (por que essa e não outra?) e as referências consultadas. Isso demonstra o raciocínio por trás do seu código.

Se você usou a IA para comparar tecnologias ou entender um conceito que levou a uma decisão, mencione isso na sua justificativa e, se possível, inclua o link da conversa como uma de suas referências. Isso demonstra um processo de pesquisa moderno e transparente.

- *Este é um documento que evoluirá ao longo do projeto. A versão completa do seu registro será incluída no **Pacote de Entrega Final da Fase 2**.*

 Dica de Mestre: Faça uma Autoavaliação!

O seu Registro de Decisões é o diário da sua jornada técnica e será avaliado apenas no final da Fase 2. Use o guia de avaliação como um **padrão de qualidade para documentar suas decisões à medida que elas acontecem**. Isso garantirá um registro rico e profissional para sua entrega final.

Passo 5: Mão na Massa! (Implementação)

O que fazer?

Chegou a hora de codificar! Com base no seu planejamento e pesquisa, comece a construir a solução, peça por peça. Crie os componentes, configure o back-end, conecte as partes.

Seu objetivo aqui:

Ter um protótipo funcional, uma primeira versão do seu projeto que já pode ser vista e testada.

💡 Por que isso é importante?

É a aplicação prática de todo o seu esforço. Ver o código se transformar em algo funcional é a parte mais recompensadora do processo!

📦 Artefatos a serem entregues

O **Repositório de Código-Fonte com o MVP (Minimum Viable Product)**. O link para seu repositório público no GitHub (ou similar) contendo a primeira versão funcional do projeto. O repositório deve ter um histórico de commits que mostre o progresso do desenvolvimento.

- *O link para seu repositório, com o histórico completo de commits do MVP, será uma peça central do seu **Pacote de Entrega Final da Fase 2**.*

💡 Dica de Mestre: Faça uma Autoavaliação!

Seu repositório é a história viva do seu projeto e será avaliado como um todo no final da Fase 2. Use o guia de avaliação como um **lembrete constante das boas práticas** (mensagens de commit claras, organização, README atualizado) durante todo o seu processo de desenvolvimento.

Passo 6: Cace os Bugs e Melhore (Validação)

O que fazer?

Seu projeto funciona... no cenário ideal. E nos outros? Tente "quebrar" sua própria aplicação. Teste todas as funcionalidades. A solução atende aos requisitos que você listou no Passo 2? Peça feedback para colegas e professores. Corrija os erros e refine a experiência do usuário.

Seu objetivo aqui:

Uma solução estável, robusta e de alta qualidade, ajustada com base em testes e feedback.

💡 Por que isso é importante?

Isso desenvolve seu pensamento crítico e seu compromisso com a qualidade. Um bom desenvolvedor não apenas cria, mas também garante que sua criação funcione bem.

📦 Artefatos a serem entregues

A **Versão Estável e Publicada da Aplicação (Deploy)**. Além do código-fonte finalizado no repositório, o principal artefato aqui é o link para a aplicação funcionando em um ambiente público (ex: Vercel para o front-end, Render para o back-end). A aplicação deve estar estável e cumprir os requisitos definidos.

- *O link para a sua aplicação publicada é a prova final do seu trabalho. Ele será incluído no **Pacote de Entrega Final da Fase 2**.*

💡 Dica de Mestre: Faça uma Autoavaliação!

A sua aplicação publicada é o resultado final de todo o seu esforço. Use o guia de avaliação como um **checklist de qualidade final**. Antes de considerar seu MVP "pronto" para ser empacotado para a entrega da Fase 2, passe por cada critério para garantir que a experiência do usuário está polida e os requisitos foram atendidos.

Passo 7: Apresente sua Obra e Reflita (Entrega)

O que fazer?

Seu trabalho não termina no último commit. Organize seu código, escreva uma boa documentação (o `README.md` é seu cartão de visitas!) e prepare uma apresentação clara. Mostre o que você construiu e, mais importante, reflita sobre a jornada:

- Quais foram os maiores desafios?
- Como você os superou?
- Qual foi o seu maior aprendizado?

Seu objetivo aqui:

Comunicar os resultados do seu trabalho de forma profissional e consolidar o aprendizado através da autoavaliação.

💡 Por que isso é importante?

Saber "vender seu peixe" é fundamental. Além disso, a reflexão é o que transforma a experiência em conhecimento duradouro, preparando você para desafios ainda maiores.

📦 Artefatos a serem entregues

1. **A Apresentação Final do Projeto:** Um conjunto de slides ou um roteiro para uma demonstração ao vivo, cobrindo a funcionalidade do projeto, a arquitetura, os desafios e os aprendizados.
 2. **A Documentação Completa no README.md:** A versão final do `README.md` do seu repositório, atualizada para incluir: uma descrição final do projeto, como executar a aplicação localmente e os links para a aplicação em produção.
- *Estes dois artefatos, que resumem sua jornada e seu produto, são os componentes finais do seu **Pacote de Entrega Final da Fase 2**.*

💡 Dica de Mestre: Faça uma Autoavaliação!

Esta é a etapa final de empacotamento do seu projeto. **Antes de submeter o seu Pacote de Entrega Final da Fase 2**, use o guia de avaliação para refinar sua apresentação e sua documentação. Uma comunicação clara e profissional é o que diferencia um bom projeto de um projeto excelente.

Sua jornada de desenvolvimento começa agora. Confie no processo, seja curioso e, acima de tudo, divirta-se construindo!

Boa codificação!

Do Caminho Reto à Espiral Ágil: Evoluindo seu Processo de Desenvolvimento

Até agora, apresentamos esses 7 passos como um caminho reto, uma jornada linear do Ponto A ao Ponto B. Essa abordagem foi intencional: ela te deu um mapa claro para que nenhuma etapa importante fosse esquecida, construindo um fundamento sólido.

Mas agora, é hora de revelar o segredo dos desenvolvedores profissionais: **o desenvolvimento de software de alta qualidade não é uma linha reta, é uma espiral.**

A Ilusão da Linha Reta

No mundo real, projetos raramente seguem um plano perfeito do início ao fim. Por quê? Porque o desenvolvimento é um processo de **descoberta**.

- Você começa a codificar (Passo 5) e descobre uma complexidade técnica que não havia previsto na sua pesquisa (Passo 4).
- Você mostra seu MVP para um amigo (Passo 6) e o feedback dele revela que um requisito (Passo 2) não era bem o que o usuário precisava.
- Você termina o projeto e imediatamente tem uma ideia para uma nova funcionalidade incrível que expande a visão original (Passo 1).

Se o processo fosse uma linha reta, você estaria preso. Mas como um profissional, você aprende a trabalhar em ciclos.

Bem-vindo ao Desenvolvimento em Espiral

Imagine seu projeto não como uma estrada, mas como um edifício. Os 7 passos que você seguiu foram a construção do **primeiro andar** — seu MVP (Minimum Viable Product). Ele é a base, funcional e essencial.

Para construir o segundo andar (adicionar uma nova funcionalidade grande), você não começa do zero. Você sobe pela **escada em espiral**, revisitando os mesmos 7 passos, mas em um novo nível:

1. **Revisite a Visão:** "Essa nova funcionalidade se alinha com o objetivo principal do projeto?"
2. **Defina Novos Requisitos:** "Quais são os requisitos *específicos* desta nova funcionalidade?"
3. **Planeje as Tarefas:** "Vamos adicionar novos cartões ao nosso quadro Kanban para isso."

4. **Pesquise Novamente:** "Precisamos de uma nova tecnologia ou biblioteca para implementar isso?"
5. **Implemente em Pequenos Ciclos:** Aqui está a grande mudança. Em vez de codificar tudo de uma vez, você trabalha em **Sprints** ou ciclos curtos.
6. **Valide Constantemente:** Você testa cada pequena parte assim que ela fica pronta.
7. **Entregue Valor Contínuo:** Você integra a nova peça ao projeto principal, tornando-o um pouco melhor a cada ciclo.

Este processo de girar através dos passos, adicionando camadas de valor ao seu projeto, é a essência de metodologias ágeis como o **Scrum**.

A Espiral Dentro da Espiral: Como os Profissionais Realmente Constroem

Até aqui, falamos da espiral em uma escala macro: construir o MVP (primeira volta), depois adicionar uma grande funcionalidade (segunda volta). Mas a mentalidade ágil é ainda mais profunda e se aplica também ao dia a dia da construção do seu primeiro MVP.

Pense nos passos 5, 6 e 7. O guia os apresenta como três grandes blocos sequenciais: primeiro você constrói tudo, depois testa tudo, depois entrega tudo. Na prática, os profissionais operam em um **micro-ciclo muito mais rápido** para cada pequena parte do projeto.

Em vez de construir a casa inteira e só depois testar a eletricidade, você:

1. **Constrói um cômodo** (implementa UMA funcionalidade, como o "cadastro de usuário").
2. **Testa a eletricidade DESSE cômodo** (valida especificamente essa funcionalidade).
3. **Integra esse cômodo à planta da casa** (mescla o código funcional à sua base de código principal).

E então você repete esse ciclo para o próximo cômodo.

Isso significa que, durante a fase de construção do seu MVP, você estará constantemente girando entre os passos 5, 6 e 7 para cada pequena tarefa do seu quadro Kanban, até que o conjunto mínimo de funcionalidades esteja completo e pronto para a entrega final.

Como isso Afeta suas Entregas e Avaliações?

Neste programa, a estrutura de avaliação foi projetada para espelhar essa realidade.

- **A Avaliação do MVP (A Primeira Volta da Espiral):** A avaliação principal que você receberá será sobre a **primeira volta completa** da espiral. Ela focará em verificar se você dominou cada um dos 7 passos fundamentais e entregou um **MVP sólido e**

bem-documentado. Este é o seu "bilhete de entrada" para provar que você construiu uma fundação de qualidade. Embora o seu processo de construção (a interação entre os Passos 5, 6 e 7) seja uma série de micro-ciclos ágeis, a avaliação final do MVP analisará o **resultado integrado** de todo esse trabalho. A qualidade do seu produto final será a evidência de quão bem você gerenciou seus micro-ciclos de desenvolvimento.

- **A Avaliação do Produto Final (A Porta Aberta):** O que acontece depois do MVP é onde você se diferencia. O projeto não "acaba" no Passo 7. Pelo contrário, ele *começa* de verdade.
 - **Quer adicionar um sistema de notificações?** Gire a espiral novamente.
 - **Quer refatorar seu código para deixá-lo mais eficiente?** Gire a espiral.
 - **Quer implementar o feedback que recebeu para uma nova funcionalidade?** Gire a espiral.
- **Esta é a porta aberta:** Embora a avaliação formal se concentre no MVP, os projetos que continuarem a evoluir através de múltiplos ciclos de melhoria serão os que verdadeiramente se destacarão. Manter seu projeto vivo, com novos commits e funcionalidades adicionadas ao longo do tempo, demonstra a mentalidade de um profissional. Este trabalho contínuo pode ser considerado em avaliações finais, na construção do seu portfólio e em processos seletivos.

Seu primeiro objetivo é completar a linha reta. Seu objetivo de carreira é dominar a espiral.

Continue construindo!

Guia de Avaliação por Pares: Aprendendo Juntos

Olá, avaliador(a)!

No mundo da tecnologia, grandes produtos são construídos por grandes equipes, não por gênios solitários. Essa colaboração não acontece por acaso; ela é sustentada por um ciclo constante de feedback, onde os membros da equipe revisam e aprimoram o trabalho uns dos outros em todas as etapas do processo.

Imagine uma equipe profissional de software:

- Antes de uma linha de código ser escrita, a **visão do produto** e os **requisitos** são debatidos e refinados por todos.
- Durante o desenvolvimento, o **plano do projeto** é revisado para garantir que todos estão na mesma página.
- Quando uma funcionalidade fica pronta, ela é testada e avaliada não apenas tecnicamente, mas também do ponto de vista do **usuário final**.
- No final, a **apresentação** do projeto para os stakeholders é ensaiada e aprimorada com o feedback dos colegas.

A avaliação por pares que você está prestes a fazer simula exatamente este ambiente profissional. Você atuará como um colega de equipe, um colaborador valioso cujo papel é oferecer uma nova perspectiva sobre os artefatos do projeto – desde a ideia inicial até a entrega final.

Nossa Etiqueta de Feedback: Construindo um Ambiente Profissional

Para que a avaliação por pares funcione, todos nós devemos nos comprometer a manter um ambiente seguro, respeitoso e focado no crescimento. Pense nesta seção como nosso código de conduta. Toda avaliação deve seguir estas quatro regras de ouro:

1. **Critique o Trabalho, Não a Pessoa**

Esta é a regra mais importante. O feedback é sempre direcionado aos artefatos e ao código, nunca ao colega.

- **Evite:** "Você não organizou bem as tarefas."
- **Prefira:** "O quadro de tarefas ficaria mais claro se os requisitos fossem quebrados em tarefas menores."

2. Seja Específico e Acionável

Feedbacks vagos não ajudam. Para que seu colega possa agir, sua sugestão precisa ser concreta.

- **Evite:** "A interface está confusa."
- **Prefira:** "Achei o fluxo de cadastro um pouco confuso na etapa de confirmação. Talvez adicionar um título claro como 'Passo 2 de 3' ajudaria o usuário a se localizar."

3. Equilibre Críticas e Elogios

A estrutura "Ponto Forte / Sugestão de Melhoria" foi criada para isso. Reconhecer o que foi bem feito é tão importante quanto apontar onde melhorar. Isso valoriza o esforço do colega e o torna mais receptivo ao feedback construtivo.

4. Seja Empático

Lembre-se: todos estão em uma jornada de aprendizado. O objetivo é ajudar seu colega a ver algo que ele talvez não tenha visto, não provar que você sabe mais. Ofereça seu feedback com a mesma gentileza com que gostaria de recebê-lo.

O que fazer com um feedback inadequado?

Se você receber uma avaliação que seja desrespeitosa, que ataque você pessoalmente ou que viole claramente nossa etiqueta, não responda ao avaliador. Utilize a opção "Sinalizar Avaliação" na plataforma para que nossa equipe de moderação possa analisar o caso. O foco aqui é sempre no aprendizado, não no conflito.

Por que Avaliar Seus Colegas? A Missão Win-Win

Ao oferecer seu feedback, você não está apenas ajudando um colega a melhorar. Você também está aprimorando suas próprias habilidades de forma profunda.

- **Para quem recebe o feedback, você oferece:**
 - Uma nova perspectiva para identificar pontos cegos no próprio trabalho.
 - Sugestões práticas para aprimorar a qualidade da entrega.
 - Motivação para continuar evoluindo.
- **Para você, como avaliador(a), você desenvolve:**
 - **Pensamento Crítico:** Para avaliar o trabalho de alguém, você precisa entender profundamente os critérios de sucesso, o que reforça seu próprio aprendizado.
 - **Novas Ideias:** Você verá diferentes formas de abordar o mesmo problema, expandindo seu repertório de soluções.
 - **Habilidades de Comunicação:** Aprender a dar feedback de forma clara, respeitosa e construtiva é uma das soft skills mais valorizadas no mercado.

Lembre-se: o objetivo não é "julgar", mas sim **construir juntos**. Seja honesto, específico e, acima de tudo, construtivo.

Recebendo e Utilizando seu Feedback: O Ciclo de Melhoria Contínua

Receber feedback é uma habilidade. O que você faz com ele define a velocidade do seu crescimento. Após receber as avaliações dos seus colegas, siga estes passos para extrair o máximo de valor do processo:

1. **Leia com a Mente Aberta:** Lembre-se da regra de ouro: o feedback é sobre o seu **trabalho**, não sobre você. Respire fundo e leia cada comentário com o objetivo de encontrar insights, mesmo que sua primeira reação seja discordar. Agradeça a quem dedicou tempo para te ajudar a melhorar.
2. **Identifique Padrões:** Um único comentário pode ser uma questão de opinião. Mas se três dos seus quatro avaliadores apontaram que seu "Documento de Visão" estava confuso, isso não é mais uma opinião – é um dado. Preste atenção especial aos pontos que se repetem.
3. **Transforme Feedback em Ação (A Etapa Crucial):** O feedback só tem valor quando leva a uma melhoria. Antes de iniciar o próximo passo do seu projeto, sua primeira tarefa é **revisar o artefato do passo anterior**.
 - Incorpore as sugestões que você considerou pertinentes.
 - Faça um novo commit no seu repositório com uma mensagem clara, por exemplo: `refactor(passo-1): incorpora feedback da avaliação por pares na visão do projeto`.
 - Este ciclo de **fazer -> receber feedback -> refinar** é o motor do desenvolvimento profissional.
4. **Aprenda a Discordar (com Reflexão):** Você não precisa aceitar 100% do feedback recebido. Se você discordar de uma sugestão, pare e reflita: "Por que eu discordo? A minha solução atual é realmente melhor ou estou apenas defendendo minha ideia original?". Esse exercício de pensamento crítico é, por si só, um grande aprendizado.

Como Avaliar: Entendendo a Escala e o Poder das Palavras

Sua avaliação terá duas partes: uma quantitativa (a escala) e uma qualitativa (o texto). Ambas são fundamentais.

A Escala de Avaliação (Likert):

Para garantir consistência, usaremos a mesma escala de 1 a 5 em todas as avaliações. Entenda o que cada nota significa:

- **5 - Concordo Totalmente:** Nível exemplar. O critério foi atendido de forma clara, completa e superou as expectativas. É um trabalho que pode servir de referência.
- **4 - Concordo:** Bom trabalho. O critério foi bem atendido, com apenas pequenos pontos que poderiam ser polidos ou melhorados.
- **3 - Neutro:** Suficiente. O critério foi atendido, mas de forma básica. Falta profundidade, clareza ou detalhe para ser considerado um bom trabalho.
- **2 - Discordo:** Incompleto ou confuso. O critério foi atendido apenas parcialmente ou de forma que gera dúvidas. Precisa de uma revisão significativa.
- **1 - Discordo Totalmente:** Não atendido. O critério não foi cumprido ou o artefato correspondente está ausente.

O Feedback Qualitativo (Obrigatório):

Lembre-se de que seu feedback escrito deve justificar as notas que você atribuiu. A escala quantitativa oferece o "o quê" (a avaliação), enquanto o texto qualitativo fornece o "porquê" (a justificativa). Se você avaliou um critério com uma nota 3 ('Neutro'), sua "Sugestão de Melhoria" deve explicar o que faltou para que o trabalho alcançasse um nível 4 ('Concordo') ou 5 ('Concordo Totalmente'). Essa conexão é o que torna o feedback verdadeiramente útil e acionável.

Os números dão uma nota, mas **as palavras geram o crescimento**. Seu feedback escrito é a parte mais valiosa da avaliação.

Lembre-se de que seu feedback escrito deve justificar as notas que você atribuiu. A escala quantitativa oferece o "o quê" (a avaliação), enquanto o texto qualitativo fornece o "porquê" (a justificativa). Se você avaliou um critério com uma nota 3 ('Neutro'), sua "Sugestão de Melhoria" deve explicar o que faltou para que o trabalho alcançasse um nível 4 ('Concordo') ou 5 ('Concordo Totalmente'). Essa conexão é o que torna o feedback verdadeiramente útil e acionável.

Sempre inclua:

1. **Um Ponto Forte:** Comece com um elogio sincero e específico. Isso mostra que você analisou o trabalho com atenção e valoriza o esforço do colega.

2. **Uma Sugestão de Melhoria:** Seja específico e ofereça uma sugestão prática. Em vez de dizer "Está confuso", tente "Acho que seu documento ficaria mais claro se você separasse os requisitos em uma lista com marcadores".

Obrigado por sua dedicação a este processo. Ao avaliar com seriedade e empatia, você contribui para criar uma comunidade de aprendizagem mais forte para todos.

Passo 1: Entenda a Missão (Diagnóstico)

Instruções para o Avaliador:

Olá! Você está prestes a avaliar o **Documento de Visão do Projeto** de um colega. O objetivo deste artefato é demonstrar uma compreensão clara e profunda do desafio proposto no início do projeto.

Seu papel é analisar o documento e responder às perguntas abaixo de forma honesta e construtiva. Lembre-se: um bom feedback ajuda o colega a crescer e também aprimora sua própria percepção sobre o projeto.

Perguntas de Avaliação para o "Documento de Visão do Projeto"

- Clareza do Problema:** O documento descreve de forma clara e concisa qual é o problema que a solução se propõe a resolver.
(Avalie se você, como leitor, consegue entender o "porquê" do projeto existir.)
[1] [2] [3] [4] [5]
- Identificação dos Usuários:** Os usuários-alvo da solução (para quem o projeto está sendo construído) são claramente identificados.
(Avalie se o autor especificou quem se beneficiará com a solução.)
[1] [2] [3] [4] [5]
- Definição do Objetivo:** O principal objetivo do projeto (o "o quê" será construído) é apresentado de forma direta e compreensível.
(Avalie se a meta principal da solução está explícita e fácil de entender.)
[1] [2] [3] [4] [5]
- Identificação das Restrições:** As principais restrições ou condições do projeto (limites de tecnologia, escopo mínimo, etc.) foram identificadas e listadas.
(Avalie se o autor demonstrou ter entendido as "regras do jogo".)
[1] [2] [3] [4] [5]
- Qualidade Geral da Análise:** No geral, o documento demonstra uma compreensão profunda e alinhada com o enunciado do desafio, conectando problema, usuário e objetivo de forma coesa.
(Avalie a impressão geral: o autor realmente "mergulhou" no problema ou apenas listou os tópicos?)
[1] [2] [3] [4] [5]

Feedback Qualitativo (Obrigatório):

Para tornar sua avaliação ainda mais útil, por favor, escreva um breve comentário.

- **Ponto Forte:** Qual foi o aspecto mais bem executado no Documento de Visão do seu colega? (Ex: "Gostei muito de como você descreveu o usuário-alvo, ficou fácil de empatizar com ele.")
- **Sugestão de Melhoria:** Qual é a principal sugestão que você daria para tornar este documento ainda mais claro ou completo? (Ex: "Acho que seria útil detalhar um pouco mais as restrições técnicas mencionadas.")

Passo 2: Crie o Mapa do Tesouro (Requisitos)

Instruções para o Avaliador:

Você está avaliando a **Lista de Requisitos (Backlog Inicial)** de um colega. O objetivo deste artefato é traduzir a visão do projeto (definida no Passo 1) em uma lista de funcionalidades e características concretas que guiarão o desenvolvimento.

Analise a lista de Requisitos Funcionais (RFs) e Não Funcionais (RNFs) e responda às perguntas abaixo. Seu feedback ajudará seu colega a construir um plano sólido.

Perguntas de Avaliação para a "Lista de Requisitos"

- 1. Distinção entre Requisitos:** O documento separa claramente os Requisitos Funcionais (RFs - o que o sistema faz) dos Requisitos Não Funcionais (RNFs - como o sistema deve ser).
(Avalie se a organização da lista está correta e fácil de entender.)
[1] [2] [3] [4] [5]
- 2. Clareza dos Requisitos Funcionais (RFs):** Os Requisitos Funcionais são descritos de forma clara, específica e orientada à ação.
(Avalie se cada RF descreve uma funcionalidade ou ação do usuário de maneira que não deixe dúvidas sobre o que precisa ser implementado.)
[1] [2] [3] [4] [5]
- 3. Relevância dos Requisitos Não Funcionais (RNFs):** Os Requisitos Não Funcionais são pertinentes ao projeto e descrevem qualidades importantes da solução (como usabilidade, segurança, performance).
(Avalie se os RNFs adicionam valor real ao projeto e não são apenas genéricos.)
[1] [2] [3] [4] [5]
- 4. Cobertura do Escopo:** A lista de requisitos parece cobrir o escopo mínimo necessário para atingir o objetivo principal do projeto, conforme definido no Passo 1.
(Avalie se, ao ler a lista, você sente que nenhuma funcionalidade essencial foi esquecida para criar a primeira versão do produto.)
[1] [2] [3] [4] [5]
- 5. Coerência com a Visão do Projeto:** A lista de requisitos como um todo é uma resposta direta e lógica ao problema e aos objetivos definidos no "Documento de Visão do Projeto" (Passo 1).
(Avalie se há uma conexão clara entre o "porquê" do projeto e o "o quê" será construído.)
[1] [2] [3] [4] [5]

Feedback Qualitativo (Obrigatório):

Para tornar sua avaliação ainda mais útil, por favor, escreva um breve comentário.

- **Ponto Forte:** Qual foi o aspecto mais bem elaborado na lista de requisitos do seu colega? (Ex: "Achei excelente como você detalhou os requisitos de segurança, isso mostra uma grande preocupação com a qualidade.")
- **Sugestão de Melhoria:** Qual é a principal sugestão para aprimorar esta lista? (Ex: "O requisito 'Gerenciar perfil' está um pouco vago. Talvez você pudesse quebrá-lo em requisitos menores como 'Editar nome', 'Trocar foto', etc.")

Passo 3: Trace a Rota (Planejamento)

Instruções para o Avaliador:

Você está avaliando o **Quadro de Gerenciamento de Tarefas** de um colega (em ferramentas como Trello, Jira ou GitHub Projects). O objetivo deste artefato é organizar os requisitos do Passo 2 em um plano de ação visual e gerenciável.

Analise a estrutura do quadro, as tarefas (cartões) e a organização geral. Seu feedback ajudará seu colega a ter um fluxo de trabalho mais eficiente.

Perguntas de Avaliação para o "Quadro de Gerenciamento de Tarefas"

1. **Estrutura do Quadro:** O quadro possui uma estrutura lógica e intuitiva (ex: colunas 'A Fazer', 'Em Andamento', 'Concluído').

(Avalie se o quadro é fácil de entender e se o fluxo de trabalho está claro.)

[1] [2] [3] [4] [5]

2. **Granularidade das Tarefas:** Os requisitos do Passo 2 foram quebrados em tarefas (cartões) pequenas e gerenciáveis.

(Avalie se as tarefas são específicas o suficiente para serem concluídas em um curto período, em vez de serem grandes e vagas como "Fazer o front-end".)

[1] [2] [3] [4] [5]

3. **Clareza das Tarefas:** Cada tarefa (cartão) tem um título claro e descritivo que resume bem o que precisa ser feito.

(Avalie se você consegue entender o objetivo de cada tarefa apenas lendo seu título.)

[1] [2] [3] [4] [5]

4. **Cobertura do Backlog:** O quadro contém tarefas que cobrem todos (ou a maioria) dos requisitos definidos no backlog inicial (Passo 2).

(Avalie se o plano de ação no quadro está completo e alinhado com o que foi especificado anteriormente.)

[1] [2] [3] [4] [5]

5. **Organização Geral:** O quadro como um todo está bem organizado, com as tarefas iniciais posicionadas corretamente na coluna 'A Fazer', demonstrando que o planejamento foi concluído.

(Avalie a impressão geral de organização e preparo para iniciar o desenvolvimento.)

[1] [2] [3] [4] [5]

Feedback Qualitativo (Obrigatório):

Para tornar sua avaliação ainda mais útil, por favor, escreva um breve comentário.

- **Ponto Forte:** Qual foi o aspecto mais bem organizado no quadro de tarefas do seu colega? (Ex: "Adorei como você usou etiquetas para diferenciar tarefas de front-end e back-end, ficou muito visual.")
- **Sugestão de Melhoria:** Qual é a principal sugestão para tornar este planejamento ainda melhor? (Ex: "A tarefa 'Implementar API' é muito grande. Sugiro quebrá-la em tarefas menores, uma para cada endpoint, como 'Criar endpoint de login', 'Criar endpoint de registro', etc.")

Passo 4: Abasteça suas Ferramentas (Pesquisa)

Instruções para o Avaliador:

Você está avaliando o **Registro de Decisões Técnicas** de um colega. Este artefato é crucial, pois documenta o *raciocínio* por trás das escolhas tecnológicas feitas durante o projeto. O objetivo é avaliar a qualidade da pesquisa e da justificativa, não se você concorda ou não com a decisão em si.

Analise as decisões registradas e responda às perguntas abaixo. Seu feedback ajudará seu colega a desenvolver uma habilidade fundamental: a comunicação de decisões de engenharia.

Perguntas de Avaliação para o "Registro de Decisões Técnicas"

- Identificação da Decisão:** Cada registro identifica claramente qual foi a decisão técnica tomada.
(Avalie se está explícito o que foi decidido, ex: "Uso da biblioteca Axios para requisições HTTP".)
[1] [2] [3] [4] [5]
- Qualidade da Justificativa:** A justificativa para cada decisão é lógica, bem fundamentada e vai além do "porque sim".
(Avalie se o autor explica os benefícios da escolha, compara com alternativas ou alinha a decisão com os requisitos do projeto.)
[1] [2] [3] [4] [5]
- Evidência de Pesquisa:** O registro cita fontes ou referências (links para documentação, artigos, tutoriais) que embasaram a decisão.
(Avalie se o autor demonstra que a decisão não foi um "chute", mas sim fruto de uma pesquisa.)
[1] [2] [3] [4] [5]
- Relevância das Decisões:** As decisões documentadas são significativas para o projeto (ex: escolha de uma biblioteca principal, padrão de arquitetura, estratégia de autenticação).
(Avalie se o autor focou em decisões importantes em vez de detalhes triviais.)
[1] [2] [3] [4] [5]
- Clareza e Organização:** O documento como um todo é bem organizado, fácil de ler e entender, mesmo para alguém que não participou do processo de decisão.
(Avalie a qualidade geral da escrita e da estrutura do documento.)
[1] [2] [3] [4] [5]

Feedback Qualitativo (Obrigatório):

Para tornar sua avaliação ainda mais útil, por favor, escreva um breve comentário.

- **Ponto Forte:** Qual foi o aspecto mais bem executado no registro de decisões do seu colega? (Ex: "Achei excelente a forma como você comparou Axios e Fetch, listando prós e contras de cada um antes de justificar sua escolha.")
- **Sugestão de Melhoria:** Qual é a principal sugestão para aprimorar este artefato? (Ex: "Para a decisão sobre JWT, seria ótimo se você incluísse um link para a documentação ou um artigo que te ajudou a entender o conceito.")

Passo 5: Mão na Massa! (Implementação)

Instruções para o Avaliador:

Você está avaliando o **Repositório de Código-Fonte com o MVP (Minimum Viable Product)** de um colega. O objetivo deste artefato é demonstrar que a primeira versão funcional do projeto foi construída, e que o processo de desenvolvimento foi registrado de forma adequada.

Sua tarefa é explorar o repositório no GitHub (ou similar), analisar o histórico de commits e, se possível, clonar e rodar o projeto localmente.

Perguntas de Avaliação para o "Repositório de Código-Fonte com o MVP"

- Organização do Repositório:** O repositório está bem organizado, com uma estrutura de pastas e arquivos lógica e compreensível.
(Avalie se é fácil encontrar os principais arquivos do projeto, como código-fonte, configurações, etc.)
[1] [2] [3] [4] [5]
- Qualidade do Histórico de Commits:** O histórico de commits conta uma história clara do desenvolvimento. As mensagens são descritivas e os commits são frequentes e de tamanho razoável.
(Avalie se as mensagens de commit são úteis, como "feat: implementa funcionalidade de login" em vez de "atualizações" ou "correções".)
[1] [2] [3] [4] [5]
- Documentação Mínima (README):** O arquivo `README.md` contém as informações mínimas para entender o projeto e, idealmente, instruções sobre como executá-lo localmente.
(Avalie se o README foi atualizado desde o Passo 1 e se ele ajuda um novo desenvolvedor a se situar no projeto.)
[1] [2] [3] [4] [5]
- Funcionalidade do MVP:** O código no repositório representa uma versão funcional do produto, mesmo que mínima. As funcionalidades essenciais do MVP parecem estar implementadas.
(Avalie se, ao inspecionar o código (ou rodar o projeto), você consegue ver que o núcleo da aplicação foi construído.)
[1] [2] [3] [4] [5]
- Qualidade do Código (Visão Geral):** O código é legível e parece seguir boas práticas básicas (nomes de variáveis claros, funções pequenas, formatação consistente).
(Avalie a primeira impressão do código. Não é preciso uma análise profunda, mas sim verificar se há um cuidado com a clareza e a organização.)
[1] [2] [3] [4] [5]

Feedback Qualitativo (Obrigatório):

Para tornar sua avaliação ainda mais útil, por favor, escreva um breve comentário.

- **Ponto Forte:** Qual foi o aspecto de maior qualidade no repositório ou no código do seu colega? (Ex: "Fiquei impressionado com a qualidade das suas mensagens de commit, elas realmente contam a história do desenvolvimento.")
- **Sugestão de Melhoria:** Qual é a principal sugestão para aprimorar o repositório ou o código? (Ex: "O projeto funciona, mas seria ótimo se você adicionasse instruções no README sobre quais variáveis de ambiente são necessárias para rodá-lo localmente.")

Passo 6: Cace os Bugs e Melhore (Validação)

Instruções para o Avaliador:

Você está avaliando a **Versão Estável e Publicada da Aplicação** de um colega. O objetivo deste artefato é demonstrar que o projeto não apenas foi construído, mas também foi testado, refinado e implantado com sucesso em um ambiente público.

Sua tarefa é acessar a aplicação através do link fornecido, testar suas funcionalidades principais e verificar se ela cumpre o que foi prometido nos Passos 1 e 2.

Perguntas de Avaliação para a "Versão Estável e Publicada da Aplicação"

- 1. Acessibilidade e Disponibilidade:** A aplicação está acessível e funcionando através do link de deploy fornecido.
(Avalie se o link abre e a aplicação carrega corretamente, sem erros imediatos.)
[1] [2] [3] [4] [5]
- 2. Cumprimento dos Requisitos Funcionais:** A aplicação implementa as funcionalidades essenciais definidas na lista de requisitos (Passo 2).
(Teste as principais funcionalidades. Elas existem e funcionam como esperado?)
[1] [2] [3] [4] [5]
- 3. Estabilidade e Tratamento de Erros:** A aplicação é estável durante o uso normal e lida de forma adequada com entradas inválidas ou erros.
(Tente "quebrar" a aplicação. O que acontece se você deixar um campo em branco? A aplicação trava ou mostra uma mensagem de erro amigável?)
[1] [2] [3] [4] [5]
- 4. Qualidade da Experiência do Usuário (UX):** A aplicação é intuitiva e fácil de usar. O fluxo de navegação é lógico e a interface é limpa e agradável.
(Avalie a sensação de usar o produto. É fácil entender o que fazer a seguir? A experiência é fluida?)
[1] [2] [3] [4] [5]
- 5. Cumprimento dos Requisitos Não Funcionais:** A aplicação demonstra atenção aos RNFs definidos (ex: se um RNF era "ser responsivo", a aplicação se adapta bem a diferentes tamanhos de tela?).
(Verifique pelo menos um RNF importante definido no Passo 2 e avalie se ele foi cumprido.)
[1] [2] [3] [4] [5]

Feedback Qualitativo (Obrigatório):

Para tornar sua avaliação ainda mais útil, por favor, escreva um breve comentário.

- **Ponto Forte:** Qual foi o aspecto que você mais gostou na aplicação final do seu colega? (Ex: "A interface ficou muito limpa e moderna, e o processo de cadastro é extremamente intuitivo.")
- **Sugestão de Melhoria:** Qual é a principal sugestão para aprimorar a aplicação? (Ex: "A funcionalidade de busca funciona bem, mas seria ótimo se houvesse um indicador de 'carregando' enquanto ela busca os resultados.")

Passo 7: Apresente sua Obra e Reflita (Entrega)

Instruções para o Avaliador:

Você está avaliando os artefatos finais de comunicação de um colega: a **Apresentação Final do Projeto** e a **Documentação Completa no README.md**. O objetivo aqui não é mais avaliar o produto, mas sim a habilidade do colega em comunicar o valor do seu trabalho e refletir sobre seu processo de aprendizagem.

Analise os slides (ou roteiro da demo) e a versão final do arquivo `README.md` no repositório.

Perguntas de Avaliação para a "Apresentação e Documentação Final"

- Clareza da Apresentação:** A apresentação (slides ou roteiro) comunica de forma clara e objetiva o propósito e as funcionalidades principais do projeto.
(Avalie se, ao ver a apresentação, você entende rapidamente o que o projeto faz e para quem ele se destina.)
[1] [2] [3] [4] [5]
- Profundidade da Reflexão:** A apresentação e/ou a documentação demonstram uma reflexão honesta sobre os desafios enfrentados e os principais aprendizados adquiridos durante o projeto.
(Avalie se o autor foi além de apenas demonstrar o produto e compartilhou insights sobre sua jornada de desenvolvimento.)
[1] [2] [3] [4] [5]
- Qualidade da Documentação (README):** O arquivo `README.md` final é uma porta de entrada completa para o projeto. Ele contém uma descrição clara, os links para a aplicação em produção e instruções detalhadas para executar o projeto localmente.
(Avalie se um novo desenvolvedor conseguiria entender e rodar o projeto usando apenas o README.)
[1] [2] [3] [4] [5]
- Profissionalismo da Entrega:** O conjunto (apresentação + documentação) demonstra cuidado e profissionalismo. O texto é bem escrito, o visual é limpo e a informação está bem organizada.
(Avalie a qualidade geral do "pacote" de entrega. Ele passa uma imagem de um trabalho bem-acabado?)
[1] [2] [3] [4] [5]
- Coerência da Narrativa:** A apresentação e a documentação contam uma história coesa e convincente sobre o projeto, conectando o problema inicial (Passo 1) com a solução final (Passo 6).
(Avalie se a entrega final fecha o ciclo do projeto de forma lógica e satisfatória.)
[1] [2] [3] [4] [5]

Feedback Qualitativo (Obrigatório):

Para tornar sua avaliação ainda mais útil, por favor, escreva um breve comentário.

- **Ponto Forte:** Qual foi o aspecto mais impressionante na apresentação ou na documentação final do seu colega? (Ex: "Achei a seção de 'desafios e aprendizados' na sua apresentação muito honesta e bem explicada.")
- **Sugestão de Melhoria:** Qual é a principal sugestão para aprimorar a forma como o projeto foi apresentado ou documentado? (Ex: "As instruções para rodar o projeto localmente no README estão boas, mas faltou mencionar a necessidade de criar um arquivo `.env`. Adicionar essa informação deixaria o guia perfeito.")